RSN Matching Engine

Purpose

The matching engine is one of the core systems in RSN.

Its job is not simply to pair two people.
Its job is to create the best possible event experience by allocating relevant, novel, and useful conversations across a live event.

This means the engine must:

- prevent bad or wasteful matches
- increase the chance of relevant conversations
- support different event types with different matching goals
- learn from feedback over time
- allow controlled fallback when perfect matches are not possible
- be understandable and operable by hosts and admins

The matching engine is therefore both:

1. an event-time action system
2. an admin-level template and rules system

What the matching button does

Matching is not always running in the background.
For live events, matching should happen when the host decides to trigger it.

Inside an event, the host should have a clear button:

Match People

When the host presses this button, the platform should:

1. take all currently eligible participants in the event lobby
2. apply the selected matching template for that event
3. generate pairings for the next round
4. allow the host or event manager to review or amend pairings if that feature is enabled
5. send users into breakout rooms when the round starts

This means matching is tied to a specific round in a specific event.
It is not a passive recommendation engine only.
It is an active live event operation.

Core principle

The engine should think in this order:

1. Hard exclusions
2. Template rules
3. User preferences
4. Event intent
5. Historical memory
6. Fairness across the room
7. Fallback logic

That order matters.

A system that chases perfect relevance without enforcing exclusions will create bad experiences.
A system that only applies rigid exclusions without fallback will strand users.
A good engine must do both.

Event types and templates

The same matching logic should not be forced onto every event.
Different event types need different templates.

Examples:

- Raw Speed Networking
- Investors meet founders
- Employers meet candidates
- Leaders meet their organisation
- Vendors meet buyers
- Peer groups
- Mentor matching
- Community networking
- Internal company networking

Each event should be linked to a **matching template**.

A template defines:

- the goal of the event
- what types of people should preferably meet

- what types of people should not meet
- what scoring logic matters most
- what exclusions are absolute
- how strong fallback is allowed to be
- whether rematching old contacts is allowed after a long cooldown
- whether host review is enabled

Matching templates in simple terms

A matching template is a reusable ruleset.

It should tell the engine:

- who this event is for
- what makes a good match in this event
- what makes a bad match in this event
- how strict the engine should be
- how much serendipity is allowed

Example: Raw Speed Networking template

Goal:
Help people meet useful new people with strong conversation potential.

High priority:

- novelty
- relevance
- mutual usefulness
- conversation quality

Strong rules:

- do not match people who already met recently
- do not match people who invited each other
- avoid same company
- avoid explicit unwanted match types

Moderate rules:

- align event intentions
- align desired counterpart types
- reward useful cross-functional matches
- allow some surprising but valid matches

Example: Investor meets founder template

Goal:
Create founder to investor conversations.

High priority:

- founder to investor pairing
- investor interest areas
- founder stage relevance
- sector relevance if desired

Strong rules:

- avoid founder to founder unless fallback
- avoid investor to investor unless fallback
- avoid repeat matches

Example: Employer meets candidate template

Goal:
Create useful hiring conversations.

High priority:

- employer to candidate pairing
- role relevance
- job status relevance
- geography or remote preference if needed

Strong rules:

- avoid employer to employer unless fallback
- avoid candidate to candidate unless fallback

User profile data for matching

The system should use structured profile data.
If users do not complete enough of their profile, they can still be matched, but the system should fall back to safer, simpler logic.

Identity

- full name
- age range
- country
- city
- timezone
- languages spoken

Professional context

- industry
- functional area
- current title
- designation
- employment status
- company name
- company URL
- LinkedIn URL
- years of experience
- company stage or size

Best designation options

- Founder
- Co founder
- Company owner
- CEO
- Leader
- Manager
- Employee
- Investor
- Advisor
- Consultant
- Board member
- Freelancer
- Student
- Job seeker
- Other

Employment status

- Employed full time
- Self employed
- Founder building company
- Investor active
- Between jobs
- Exploring opportunities
- Student
- Retired
- Other

Matching preferences

Users should not only say who they are.
They should also say who they want to meet and who they do not want to meet.

Who would you like to meet

Use structured categories first.
Optional free text second.

Example categories:

- Founders
- Investors
- Operators
- Commercial leaders
- Technical leaders
- Advisors
- Employers
- Talent
- Potential clients
- Potential partners
- People from my industry
- People outside my industry
- People in my city
- People globally
- People at a similar stage
- People ahead of me
- People earlier in their journey

Who would you prefer not to meet

Example categories:

- People trying to sell to me
- Recruiters
- Investors
- Early stage founders
- People from my own company
- Direct competitors
- People outside my industry
- People only looking for jobs
- No strong preference

Match style or conversation type

Users can also state what kind of conversations they want:

- Tactical

- Strategic
- Reflective
- Commercial
- Fundraising
- Hiring
- Partnerships
- Peer support
- Friendship
- Accountability
- Career opportunities
- Mentorship
- Learning

Event level intention

This should be separate from the permanent profile.

A person may want one thing at one event and something else at another.

Before or during event check in, ask:

What is your intention for this event?

Example options:

- Meet useful new people
- Find potential clients
- Find partners
- Find investors
- Meet founders
- Get advice
- Give advice
- Find a job
- Hire talent
- Explore opportunities
- Build friendships
- Get perspective
- Stay connected to the community
- Other

Also ask:

How open are you to unexpected matches?

- Very open
- Somewhat open
- Only highly relevant matches

This gives the template a controlled serendipity input.

Hard exclusions

These rules should block a match unless the system enters a specific last resort mode.

1. Never match the same people again inside the same event
2. Never match users who directly invited each other
3. Never match blocked users
4. Never match users with explicit mutual avoidance
5. Never match users from the same company if that rule is enabled for the event
6. Never match users already paired in the current round
7. Never match users who are in violation status or manually restricted

Long memory rule

The platform should remember who has met whom.

Suggested default logic:

- same event: never again
- across events: do not rematch for 12 months by default
- after 12 months: rematch becomes technically possible, but heavily penalized and only used when no better option exists or when the template explicitly allows it

Match storage, analytics, and learning loop

Everything the engine does should be stored.
Not only the final ratings.
The full matching history matters.

This is important for three reasons:

1. to improve future matching quality
2. to analyse what creates the best experience
3. to build new RSN products on top of the relationship graph over time

What should be stored for every match

For every attempted or completed match, the system should store:

- event ID
- round ID

- template used
- both user IDs
- whether the match was automatic or manually overridden
- match score
- key reasons or factors behind the score
- confidence level
- whether the match actually happened
- whether either user left before or during the conversation
- conversation duration
- post conversation ratings
- meet again signal
- timestamp

What should be analysed over time

The system should make it possible to analyse:

- which templates perform best
- which fields are most predictive of good conversations
- which match types lead to strong ratings
- which users or profile types are frequently unmatched
- which fallback levels perform poorly
- whether host overrides outperform automatic matching
- whether certain event formats create better repeat attendance or stronger follow up behaviour

This allows RSN to continuously improve the engine rather than treating it as static logic.

The long term value of match storage

Over time, RSN should build a relationship graph.

This graph can show:

- who has met whom
- which conversations went well
- which relationship types are productive
- who tends to help whom
- who tends to want to meet again
- which users are likely to become meaningful future connections

This becomes one of the platform's strongest long term assets.

Conversation ratings and learning loop

After every conversation, users should rate the interaction.

Minimum rating inputs:

1. How was the conversation?
2. Would you like to meet this person again?

Optional later additions:

- Was this relevant to your goal?
- Did this person try to sell to you?
- Would you recommend this person to others?

These ratings should not instantly override matching.
But they should become part of the system memory and template scoring.

How the ratings should be used

- positive rating increases future compatibility score in relevant contexts
- negative rating decreases future compatibility score
- meet again creates a relationship signal for post event follow up, future pods, or reconnection suggestions
- repeated poor ratings may help detect bad actors or low quality participants
- strong ratings can improve recommendation quality over time

Important:
The event engine and the relationship engine are related, but not identical.

In live speed networking, the first job is to create great real time pairings.
After that, ratings can improve later logic.

Matching score layers

Each possible pair should be scored.

A simple way to think about it:

Pair score = eligibility + compatibility + diversity + fairness + fallback confidence

1. Eligibility

This is pass or fail.

If any hard exclusion applies, the pair is not eligible.

2. Compatibility

This is the core relevance score.

It should include things like:

- event intention alignment
- desired counterpart alignment
- professional relevance
- designation compatibility
- industry relevance or productive contrast
- language fit
- geography or timezone fit if relevant
- offer and need alignment if used

3. Diversity

This avoids repetitive event experiences.

It should reward:

- new types of people
- avoiding same designation repeatedly
- avoiding same industry repeatedly when appropriate
- broader exposure when the template wants it

4. Fairness

This balances the room.

The engine should not simply give all best matches to the most attractive profiles. It should optimize total room quality.

This means considering:

- whether some users have already had weak matches in earlier rounds
- whether some users have very few options left
- whether one user is consuming too many strong pairings
- whether some people are repeatedly being stranded

5. Fallback confidence

When no strong match exists, the system should still choose the safest and best available option.

Fallback ladder

If ideal matching is not possible, the engine should step down in layers.

Level 1

Full template scoring using complete profile, preference, intent, and history data

Level 2

Partial scoring using whatever profile data exists

Level 3

Basic safe matching only:

- not already met
- not invited by each other
- not blocked
- not same company if relevant
- not matched already in this event round

Level 4

Random among eligible people

Level 5

Long cooldown rematch only if the template allows it and there are no valid alternatives

This means random should never be truly random.
It should still be random within safe constraints.

Incomplete profiles

If a user has not filled out their profile properly, they should not be excluded from the event.
They should simply receive lower precision matching.

Suggested rule:

- complete profile = full scoring
- partial profile = partial scoring
- minimal profile = safe fallback matching

This avoids dead users in the system while still rewarding profile completion.

AI template wizard

The platform should include an AI wizard that helps admins, and later users, create their own matching templates.

This matters because most people know what kind of event they want to create, but they do not know how to define matching logic in system language.

The wizard should allow someone to describe the event in plain language and then convert that into a structured template.

Example inputs

- "Create an event where early stage founders meet investors"
- "Make a networking event for leaders who want peer advice, but avoid sales people"
- "Create a pod where operators meet founders and nobody should meet the same person twice in 6 months"

The wizard should help define

- event goal
- participant types
- preferred pairings
- excluded pairings
- scoring priorities
- rematch rules
- fallback rules
- exploration level
- whether host review is enabled

Build on existing templates

Users should also be able to start from an existing template and adapt it.

Examples:

- duplicate Raw Speed Networking and make it stricter
- use Investor Meets Founder as a base and change geography priority
- use Hiring Event as a base and add role matching requirements

This means templates should be modular and reusable, not hard coded one offs.

Admin interface requirements

The admin side of matching is critical.
This is not a hidden technical feature.
It is a major operational and strategic control system.

Admins should be able to:

- create, edit, duplicate, archive, and activate matching templates
- define template goals
- set hard exclusion rules
- set scoring priorities
- set cooldown periods
- define whether host review is enabled
- define fallback strictness
- define rematch permissions
- define event types and assign templates to them
- inspect round outcomes and quality metrics
- inspect rating trends
- inspect violation patterns
- view users who are frequently unmatched or poorly matched

Template controls should include

- template name
- description
- event use case
- primary matching goal
- hard exclusions
- scoring weights
- exploration level: low, medium, high
- rematch cooldown duration
- fallback behavior
- whether same company matching is allowed
- whether invite relationships are blocked
- whether host can amend pairings before launch

Host and event manager controls

At the event level, hosts or event managers should be able to:

- see who is in the lobby and is eligible for matching
- press Match People
- preview pairings if allowed
- swap or override pairings manually if enabled
- see who could not be confidently matched
- decide whether to include low confidence or fallback matches
- start round
- send everyone to breakout rooms
- handle edge cases in real time

User generated matching for pods

A future or advanced feature is that users can create their own pods and define their own matching logic with AI assistance.

This means the system architecture should already anticipate user level templates, not only admin level templates.

A pod creator may want to say things like:

- match founders with investors only
- avoid same industry matches
- prioritize people who want accountability
- create triads instead of pairs
- prioritize local city matches
- do not match anyone who has already met in the past 6 months

The AI layer could help translate natural language into template rules.

Example:

User says:
"I want a pod where early stage founders meet operators and not investors, and nobody should meet the same person twice within 6 months."

System converts that into a custom matching template.

This should be treated as future compatible architecture.
Even if V1 only supports admin templates, the data model should not block user generated templates later.

Ongoing platform matching and new friend discovery

Once RSN has enough users on the platform, matching should not be limited to live events.

The same underlying engine should also support ongoing relevance based introductions across the platform.

This becomes possible when users state:

- what they want to buy
- what they want to sell
- what they need
- what they can offer

- their intentions
- their reasons for joining
- who they want to meet
- who they do not want to meet

At that stage, the system can begin suggesting potential new friends, collaborators, clients, partners, mentors, hires, or other meaningful connections.

This should not feel like aggressive lead generation.
It should feel like intelligent human relevance.

In practice, this means the engine can evolve from:

- event matching

to also supporting:

- relationship recommendations
- connection suggestions
- pod formation
- buyer seller relevance
- accountability matches
- friendship and peer support discovery

The phrase "new friends" should be understood broadly as meaningful new human connections, not only social friendship.

Premium and paid matching modes

A future or advanced feature is that users can create their own pods and define their own matching logic with AI assistance.

This means the system architecture should already anticipate user level templates, not only admin level templates.

A pod creator may want to say things like:

- match founders with investors only
- avoid same industry matches
- prioritize people who want accountability
- create triads instead of pairs
- prioritize local city matches
- do not match anyone who has already met in the past 6 months

The AI layer could help translate natural language into template rules.

Example:

User says:
"I want a pod where early stage founders meet operators and not investors, and nobody should meet the same person twice within 6 months."

System converts that into a custom matching template.

This should be treated as future compatible architecture.
Even if V1 only supports admin templates, the data model should not block user generated templates later.

Premium and paid matching modes

RSN should also support premium event modes where participants pay for higher control or higher quality matching.

Premium curated choice mode

One strong premium concept is:

A participant can review the list of event participants and choose up to 12 people they would most like to meet.
They still only meet 5 during the event, but the system gives priority to those preferences when possible.

This should work like this:

- participant selects up to 12 preferred people before matching closes
- the system treats these as high preference targets, not absolute guarantees
- the engine still checks mutual validity, template rules, and room fairness
- final meeting set is optimized across the room

This is powerful because it lets premium users express strong intent without breaking the overall event system.

Ongoing rolling matching mode

Another premium or special mode is ongoing matching.

Instead of one fixed event round structure only, the system can keep matching relevant people in sequence for a set duration.

Example:

- event runs for 60 or 90 minutes
- each conversation lasts a defined number of minutes
- when one conversation ends, participants are continuously reallocated to the next relevant available match

This can be useful for:

- large conferences
- trade matchmaking
- premium networking sessions
- buyer seller events
- talent market events

This mode requires additional logic for live availability, queueing, and fast rematching, but it should be supported conceptually in the engine design.

Edge cases that must be handled

Odd number of participants

Options:

- create one host managed trio if the template allows trios
- keep one user in lobby and prioritize them first next round
- allow manual host intervention

User leaves before matching

- remove from eligible pool immediately
- rerun matching if necessary before round starts

User leaves after matching but before breakout starts

- mark their partner as unmatched
- reallocate if possible
- otherwise return that user to lobby or waiting state

User leaves during breakout

- log event
- return remaining user to lobby or support flow
- prioritize them for next round
- optionally provide a message explaining what happened

User is alone in breakout

System should never silently leave them there without logic.

Possible responses:

- rematch live if possible

- move them back to lobby
- let host join temporarily
- place them first in next round priority queue

Insufficient valid matches

- use fallback ladder
- clearly mark low confidence matches internally
- allow host decision where appropriate

Late joiners

- keep in lobby until next matching cycle
- optionally include them in current matching run only if timing allows

Manual overrides

- log all manual changes for later review
- distinguish auto match from manual match in analytics

Best experience objective

The engine should optimize for this order of outcomes:

1. No obviously bad matches
2. No repeated wasted matches
3. High relevance
4. Novelty
5. Balanced room experience
6. Controlled serendipity

If the engine only optimizes for similarity, events become predictable and flat.
If it only optimizes for randomness, events become noisy and weak.
The right design is controlled serendipity inside clear rules.

Simple operating explanation for non technical people

The matching engine works like this:

1. Every event has a matching template
2. Every user has a profile, preferences, and event intention
3. When the host presses Match People, the system looks at who is available right now
4. It removes any pairs that should not happen

5. It scores all valid pairings according to the event template
6. It creates the best overall set of pairings for the room, not just the best single matches
7. If strong matches are not available, it uses controlled fallback logic
8. After conversations, ratings feed back into the system for future improvement

Is this the right product design for RSN

Broadly, yes.
But the important refinement is this:

RSN should not think of itself as having one matching algorithm.
It should think of itself as having a configurable matching operating system.

That is stronger and more future proof.

A good design for RSN is therefore:

Layer 1: Core matching engine

This handles:

- exclusions
- scoring
- fairness
- fallback
- history
- round creation

Layer 2: Templates

This defines event specific matching logic.

Layer 3: Live event controls

This allows hosts and event managers to trigger and manage matching during events.

Layer 4: Analytics and learning

This stores all matching data and helps improve quality over time.

Layer 5: AI template creation

This makes the system usable by non technical people.

Layer 6: Platform relationship graph

This allows RSN to grow beyond events into ongoing meaningful introductions.

That is likely the right long term architecture.

What should be avoided is building this as one rigid event feature only.
If it is built too narrowly, RSN will outgrow it quickly.

Recommended V1 scope

To keep the first version strong and buildable, V1 should focus on:

- matching templates
- hard exclusions
- event intention
- who users want to meet
- who users do not want to meet
- designation compatibility
- industry relevance
- no repeat logic
- no invited person logic
- conversation ratings
- fallback ladder
- host trigger button
- basic manual override

Later versions can add:

- company and LinkedIn scraping
- deeper semantic profile enrichment
- AI generated template creation
- user created pod templates
- dynamic relationship graph intelligence
- more advanced prediction from historical ratings

Recommended V1 scope

To keep the first version strong and buildable, V1 should focus on:

- matching templates
- hard exclusions
- event intention
- who users want to meet
- who users do not want to meet
- designation compatibility

- industry relevance
- no repeat logic
- no invited person logic
- conversation ratings
- match storage
- fallback ladder
- host trigger button
- basic manual override
- analytics foundation

Later versions can add:

- company and LinkedIn scraping
- deeper semantic profile enrichment
- AI generated template creation
- user created pod templates
- dynamic relationship graph intelligence
- buyer seller recommendation matching
- premium participant preference selection
- rolling ongoing matching modes
- more advanced prediction from historical ratings

Bottom line

This feature should be understood as a live matching engine with memory, templates, and controls.

It is a core product system.
Not a small utility.

It should be built so that:

- admins can shape the logic
- hosts can operate it live
- users can improve results through their profiles and feedback
- the platform can support many event formats, not only one
- the system gets better over time without becoming rigid

The matching engine is one of the clearest product differentiators in RSN.